



南開大學
Nankai University

南 开 大 学

计 算 机 学 院

操作系统实验报告

Lab8: 文件系统

年级：2023 级

姓名：康志杰，王春晖，彭浩然

2026 年 1 月 1 日

目录

一、 实验目的	1
二、 实验原理	1
三、 实验过程	1
(一) 练习 0	1
(二) 练习 1	2
(三) 练习 2	3
(四) challenge1	6
(五) challenge2	7
四、 实验总结	7
(一) 实验核心知识点与 OS 原理对应分析	7
(二) OS 原理中重要但实验未涉及的知识点	8
(三) 总结	8

一、 实验目的

1. 了解文件系统抽象层-VFS 的设计与实现
2. 了解基于索引节点组织方式的 Simple FS 文件系统与操作的设计与实现
3. 了解“一切皆为文件”思想的设备文件设计
4. 了解简单系统终端的实现

二、 实验原理

文件系统的核心首要任务，是将持久存储设备的块读写抽象为进程可调用的统一接口，这一包装依赖超级块、inode 等元数据支撑。

ucore 文件系统仿 UNIX 设计，以 VFS 为统一入口：将文件操作抽象为 inode 级接口，上层系统调用不依赖具体文件系统（如 SFS），实现不同文件系统与设备文件的行为一致性。整体分为四层协作：

1. 用户态库 / 系统调用层：转换用户请求为内核操作；
2. VFS 层：负责路径解析、inode 管理与操作转发；
3. SFS 层：处理磁盘布局与 inode / 目录项语义（采用 4KB 块，通过 direct/indirect 指针实现混合索引，适配大小文件存储）；
4. 设备层：将 disk0、stdin 等封装为设备文件，提供统一读写接口，体现“一切皆文件”（终端输入通过定时中断写入 stdin 缓冲区，供 read 系统调用读取）。

此外，Lab8 核心还包括“基于文件系统的 exec”：程序以普通文件形式存储，exec 需通过 VFS 打开文件、读取 ELF 信息，将程序段映射到进程页表，这一机制改变了进程的诞生与运行方式，牵动 proc.c、do_execve 等进程相关模块的实现。

三、 实验过程

（一） 练习 0

首先对前面实验的代码进行补充，并且完成本次实验对前面代码的更新。这次实验聚焦于文件系统，在 proc_struct 结构体中新增了 struct files_struct *filesp，该属性是进程相关文件的指针。所以在 alloc_proc 函数中需要将该指针初始化为 null。

接着在 pro_run 函数中需要在上下文切换之前刷新 tlb，发生进程切换的时候 tlb 表里存储的是上一个进程的页表项，此时 lsatp 函数已经切换到下一个进程的页表，所以 tlb 中的缓存已经失效，此时需要调用函数 flush_tlb 刷新 tlb。其余部分就是将前面实验的代码进行填充，在这里就不过多解释。

```
1 void proc_run(struct proc_struct *proc)
2 {
3     if (proc != current)
4     {
5         bool flags=0;
6         struct proc_struct*pre=current,*next=proc;
7         local_intr_save(flags);
8         current=proc;
9         lsatp(proc->pgdir); //修改satp寄存器中的值
10        flush_tlb(); //刷新tlb
```

```

11         switch_to(&pre->context,&next->context);
12         local_intr_restore(flags);
13     }
14 }

```

最后在 do_fork 函数中父进程要 fork 出子进程，需要在 do_fork 函数中调用 copy_files 函数将父进程的相关文件资源复制给子进程。

```

1 if (copy_files(clone_flags, proc) != 0)
2     {
3         goto bad_fork_cleanup_fs;
4     }

```

(二) 练习 1

该练习需要完成 sfs_io_nolock 函数的功能，该函数主要是向磁盘块完成读写操作。但是每次读写并不是整块读写，所以就要针对不同情况进行处理。首先可以观察到如果当 offset 和块不对齐的话，就要从块的当前位置 blkoff 往后读写，所以以下代码就是争对这种情况，在 if 分支里第一步计算本次需处理的数据长度 size，若剩余待处理块数 nblks 不为 0，说明还有后续完整块需处理，本次仅处理当前块中从偏移位置到块末尾的部分 SFS_BLKSIZE - blkoff。若 nblks 为 0，说明当前是最后一个待处理块，仅处理从偏移位置到操作结束位置 endpos 的剩余数据。

随后调用 sfs_bmap_load_nolock 通过块号 blkno 获取对应的 inode 节点 ino，找到块号与物理存储节点的映射；再调用 sfs_buf_op 完成实际数据读写。从 inode 对应的物理块中，从 blkoff 偏移位置读或写 size 长度的数据。然后如果后续没有需要处理的块，则直接 goto 到 out 那里返回即可，否则就要对 offset, nblks, blkno 进行更新。

```

1 //处理第一个块 (offset 未对齐)
2     if (blkoff != 0) {
3         size = (nblks != 0) ? (SFS_BLKSIZE - blkoff) : (endpos - offset);
4         if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) goto
5             out;
6         if ((ret = sfs_buf_op(sfs, buf, size, ino, blkoff)) != 0) goto out;
7         alen += size;
8         buf = (char *)buf + size;
9         offset += size;
10        // 如果只处理了部分块，则直接返回
11        if (nblks == 0) {
12            goto out;
13        }
14        nblks--;
15        blkno++;
16    }

```

接着就要处理完整块的读写操作，首先单个完整块的大小为文件系统块大小 SFS_BLKSIZE，然后通过循环持续处理剩余完整块。每次循环先调用 sfs_bmap_load_nolock，将当前逻辑块号 blkno 映射为物理 inode 节点号 ino。接着调用 sfs_block_op，以物理 inode 号为标识，一次性处理 1 个完整块的读写操作。之后更新变量 alen, buf 等。

```

1 //处理中间完整块

```

```

2      uint32_t full_block_size = SFS_BLKSIZE;
3      while(nblks > 0) {
4          //获取当前逻辑块对应的物理块号
5          if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) goto
              out;
6          //传入物理块号ino, 且每次只处理1个完整块
7          if ((ret = sfs_block_op(sfs, buf, ino, 1)) != 0) goto out;
8          //更新统计和指针
9          alen += full_block_size;
10         buf = (char *)buf + full_block_size;
11         offset += full_block_size;
12         blkno++;
13         nblks--;
14     }

```

最后先判断当前操作偏移 offset 是否未到结束位置 endpos, 若是则进入最后一个块的处理。计算需处理的数据长度 size 为结束位置与当前偏移的差值。通过 sfs_bmap_load_nolock 将当前逻辑块号映射为物理 inode 号, 再调用 sfs_buf_op 从物理块起始位置读写指定长度数据。最后累加已处理数据长度 alen, 完成整个数据传输流程的收尾。此时 alen 就是实际读取的字节长度。

```

1 //处理最后一个块 (endpos未对齐)
2     if (offset < endpos) {
3         size = endpos - offset;
4         if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) goto
            out;
5         if ((ret = sfs_buf_op(sfs, buf, size, ino, 0)) != 0) goto out;
6         alen += size;
7     }

```

(三) 练习 2

练习 2 要完成 load_icode 函数相关逻辑代码, 该函数与前面的不同点在于本次实验需要从磁盘中读取 ELF 文件, 而且需要将传入的参数 argc 以及 kargv 压入用户栈中。接着就按照这两个不同点进行分析。从磁盘中读取 elf 文件内容是利用函数 load_icode_read, 此时未知 elf 文件大小, 没办法将整个文件读取出来, 所以调用函数 load_icode_read 读取 elf 函数头部 elf。接着会验证魔数 elf.e_magic, 如果魔数不对直接返回。

```

1      struct elfhdr elf;
2      if ((ret = load_icode_read(fd, &elf, sizeof(struct elfhdr), 0)) != 0)
3      {
4          goto bad_elf_cleanup_pgdir;
5      }
6      // 验证ELF魔数
7      if (elf.e_magic != ELF_MAGIC)
8      {
9          ret = -E_INVALID_ELF;
10         goto bad_elf_cleanup_pgdir;

```

11

}

接着就需要读取程序头表，`elf.e_phnum` 是程序段头表的条目数量，`ph` 是程序段头表指针。`load_icode_read` 将表项读取出来，接着就在 `for` 循环中对各个段进行处理，和前面实验实现过程一样。在拷贝各个段的时候也需要调用函数 `load_icode_read` 进行读取。

```

1 // 读取程序段头表
2 struct proghdr *ph = kmalloc(sizeof(struct proghdr) * elf.e_phnum);
3 if (ph == NULL)
4 {
5     ret = -E_NO_MEM;
6     goto bad_elf_cleanup_pgdir;
7 }
8 if ((ret = load_icode_read(fd, ph, sizeof(struct proghdr) * elf.e_phnum,
9     elf.e_phoff)) != 0)
10 {
11     goto bad_ph_cleanup;
12 }
13 struct Page *page;
14 uint32_t vm_flags, perm;
15 struct proghdr *ph_end = ph + elf.e_phnum;
16 for (; ph < ph_end; ph++)
17 {
18     // ...
19     while (start < end)
20     {
21         // ...
22         if ((ret = load_icode_read(fd, buf, size, ph->p_offset + (start -
23             ph->p_va))) != 0)
24         {
25             kfree(buf);
26             goto bad_cleanup_mmap;
27         }
28         // ...
29     }
30 }

```

第二处不同之处在于这次实验需要将参数 `argc` 以及 `kargv` 压入栈中。代码首先计算 `argv` 指针数组的总空间 `uargv_size`，即参数个数乘以指针长度。接着汇总栈空间总需求：包含指针数组空间、所有参数字符串总长度 `total_string_len`，并基于用户栈顶 `USTACKTOP` 计算出栈空间起始地址 `current_sp`，强制将地址对齐到 16 字节。

接着代码划分出栈中三个区域，`uargc_ptr` 指向 `argc` 存储位置，`uargv_ptr` 指向 `argv` 指针数组起始位置，`ustr_ptr` 指向参数字符串存储区域。先将 `argc` 值写入对应位置，再通过循环逐个处理参数。为每个 `uargv[i]` 赋值为栈中对应字符串的起始地址，同时将内核态的参数字符串 `kargv[i]` 拷贝到用户栈的字符串区域，并更新偏移量。这时参数已经成功压入栈中。

```

1 // 计算 argv 指针数组所需空间
2 uint32_t uargv_size = argc * sizeof(char *);

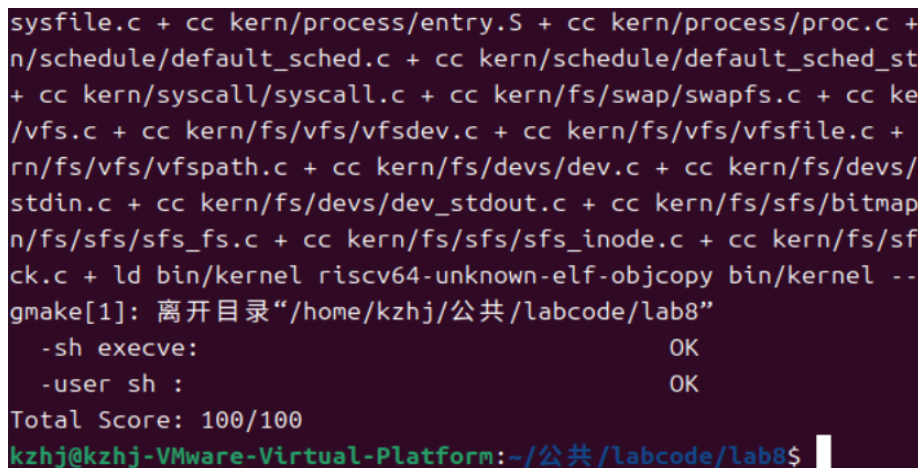
```

```

3 // 计算总共需要减去的栈空间, 并进行对齐
4 uintptr_t stack_needed = sizeof(long) + uargv_size + total_string_len;
5 uintptr_t current_sp = USTACKTOP - stack_needed;
6 current_sp -= (current_sp % 16); // 强制对齐到16字节
7 //定义三个区域的起始地址
8 int *uargc_ptr = (int *)current_sp;
9 char **uargv_ptr = (char **)(current_sp + sizeof(long));
10 char *ustr_ptr = (char *)(current_sp + sizeof(long) + uargv_size);
11 //填充内容
12 *uargc_ptr = argc; // 压入 argc
13 uint32_t offset = 0;
14 for (int i = 0; i < argc; i++) {
15     uargv_ptr[i] = ustr_ptr + offset; // 设置 uargv[i] 指向用户栈上的字符串
16     strcpy(uargv_ptr[i], kargv[i]); // 拷贝字符串到用户栈
17     offset += strlen(kargv[i]) + 1;
18 }

```

到此本次实验的代码已经实现完毕, 接下来进行验证, 首先在终端输入 `make grade` 得到如图1所示结果, 得分为 100, 说明已经达到测试目标。



```

sysfile.c + cc kern/process/entry.S + cc kern/process/proc.c +
n/schedule/default_sched.c + cc kern/schedule/default_sched_st
+ cc kern/syscall/syscall.c + cc kern/fs/swap/swapfs.c + cc ke
/vfs.c + cc kern/fs/vfs/vfsdev.c + cc kern/fs/vfs/vfsfile.c + c
rn/fs/vfs/vfspath.c + cc kern/fs/devs/dev.c + cc kern/fs/devs/d
stdin.c + cc kern/fs/devs/dev_stdout.c + cc kern/fs/sfs/bitmap
n/fs/sfs/sfs_fs.c + cc kern/fs/sfs/sfs_inode.c + cc kern/fs/sf
ck.c + ld bin/kernel riscv64-unknown-elf-objcopy bin/kernel --s
gmake[1]: 离开目录"/home/kzhj/公共/labcode/lab8"
-sh execve: OK
-user sh : OK
Total Score: 100/100
kzhj@kzhj-VMware-Virtual-Platform:~/公共/labcode/lab8$

```

图 1: make grade 测试结果

最后输入 `make qemu` 观察是否可以正常运行用户程序。得到如图2所示结果, 观察到 `sh` 脚本已经运行起来了, 接着输入程序名 `hello`, `hello` 程序正常运行并输出内容显示运行成功。接着测试 `divzero` 也正常运行。之后又测试了 `disk0` 目录下的其他指定文件结果显示均可以正常运行, 说明本次实验已经达到预期目标。

```

sfs: mount: 'simple file system' (106/11/117)
vfs: mount disk0.
++ setup timer interrupts
kernel_execve: pid = 2, name = "sh".
user sh is running!!!
Hello world!!.
I am process 3.
hello pass.
value is -1.
user panic at user/divzero.c:9:
    FAIL: T.T

error: -10 - panic failure
$

```

图 2: make qemu 测试结果

(四) challenge1

我把 Pipe 看成“特殊文件”的原因, 是这样可以最大化复用 ucore 现有的 fd/file/inode 框架: 用户仍然通过 read(fd) / write(fd) 访问, 内核仍然通过 file->node->in_ops->vop_read/write 分发, Pipe 只需要实现自己的一套 inode_ops (或 file_ops) 即可。这对应 OS 原理里“统一抽象 + 复用机制”的思想: Pipe 的本质是 IPC, 但接口形态仍然是文件。

核心数据结构我会设计为“内核环形缓冲区 + 读写端引用计数 + 同步原语”。为了让设计落在 ucore 的同步模型里, 我会显式使用 semaphore + wait_queue 的组合:

读端: 缓冲区为空则睡眠, 直到有数据或写端全部关闭 (返回 EOF)

写端: 缓冲区满则睡眠, 直到有空间或读端全部关闭 (写触发 SIGPIPE/返回 EPIPE 的语义可以简化为返回错误) 为了把 Pipe 接入 VFS, 我有两种“实现路径”, 都能在 ucore 框架里自治:

第一种是“设备式 inode”: 像 stdin/stdout 那样做一个 pseudo device, 把 d_io() 指向 pipe_io, 再用 vfs_dev_t 挂到 vdev_list, 让路径例如“pipe:”能 open; 但 Pipe 的难点是每次 pipe() 都要创建一个“新的实例”, 因此纯设备模式不够自然。

第二种 (更贴近 UNIX) 是“匿名 inode + 两个 file 对象”: pipe() 直接在内核里分配一个 pipe_t, 再创建两个 file (或两个 fd) 引用同一 pipe_t, 一个标记为可读, 一个标记为可写。这个方案不依赖目录树, 不需要路径名, 更符合 pipe 的匿名性。

因此我会给 pipe 系统调用 (或用户库接口) 定义语义为: 创建一个 pipe_t, 并返回两个 fd。它们共享同一内核对象, 但 open_flags 不同。

```

1 int sys_pipe(int fd[2]); // fd[0]=read end, fd[1]=write end

```

对应的 file/inode 关联, 我会让 file->node 指向一个“pipe inode” (可以是 inode->in_info 存 pipe_t*), 并在 inode_ops 里实现 vop_read/vop_write/vop_close。其中 close 需要减少 readers/writers 计数, 并在计数归零时释放 pipe_t, 同时唤醒对端避免死睡眠 (例如写端全关后唤醒读端返回 EOF, 读端全关后唤醒写端返回 EPIPE/错误)。

同步互斥部分我会写得更“像真实系统”: 所有对 rpos/wpos/readers/writers 的修改都在 lock 保护下进行; 读写在条件不满足时进入 wait_queue; 唤醒时必须在状态更新后进行, 避免丢唤醒。ucore 里 wait_queue 的使用方式与 stdin 的实现类似 (有 sleep 与 wakeup 逻辑), Pipe 完全可以复用这一模型。

(五) challenge2

我在设计链接机制时，先把概念与磁盘层对应关系理清：

硬链接 (hard link)：多个目录项 (dentry/dir entry) 指向同一个 inode (同一个 ino)。删除文件名只是减少一个引用计数；只有当链接计数为 0 且没有进程打开该文件时，才释放 inode 与数据块。

软链接 (symbolic link)：本质是一个独立 inode (独立文件)，其数据内容是“目标路径字符串”。访问软链接时，需要在路径解析阶段把它替换为目标路径继续解析。

ucore 的 SFS 已经有 nlinks 字段 (磁盘 inode 里就有)，这为硬链接提供了天然落点：硬链接本质上只需要“创建一个新的目录项 entry 指向同一 ino，并把 inode->nlinks++”。

因此硬链接的关键不在于数据结构新增，而在于：把“目录项操作”与“回收策略”补齐。我会定义一个系统调用语义 (或内核接口)：

```
1 int sys_link(const char *oldpath, const char *newpath); // hard link
2 int sys_unlink(const char *path); // remove a name
```

sys_link 的过程应该是：解析 oldpath 得到 inode；在 newpath 的父目录中创建一个新的目录项，ino 指向 oldpath inode；随后把 old inode 的 nlinks++ 并写回 (标记 dirty)。并发上需要对目录 inode 加锁、对目标 inode 加锁，且要注意锁顺序避免死锁 (例如按 ino 编号排序加锁)。sys_unlink 则相反：删除目录项，将 inode->nlinks-，若 nlinks==0 且 open_count==0，才真正回收数据块与 inode (对应 SFS 的 truncate/free 逻辑)。

软链接则需要新增一种 inode 类型 (或复用现有 type 字段扩展一个 SFS_TYPE_SYMLINK)，并把“目标路径”存到该 inode 的数据块里。其接口语义可以是：

```
1
2 int sys_symlink(const char *target, const char *linkpath);
3 int sys_readlink(const char *linkpath, char *buf, size_t buflen);
```

实现 sys_symlink 的核心是：在 linkpath 位置创建一个新 inode (type=SYMLINK)，文件内容写入 target 字符串。sys_readlink 则只是读取该 inode 的内容并返回，不进行路径跳转。

软链接最关键的是路径解析阶段的处理。在 VFS 的 lookup (或 path walk) 中，当解析到一个路径分量对应的 inode 是 symlink 时，应当读取其内容 (target)，并把剩余路径拼接到 target 后继续解析；同时必须限制最大解析深度 (例如 8 或 16)，防止 a->b->a 的循环链接导致无限递归。这一点在 OS 原理课里通常作为“名称解析与安全性”讨论，在实验里如果你写出来，会非常加分，因为它体现了你不是只停留在“存一个字符串”，而是考虑了真正的路径语义与边界条件。

并发与一致性方面，链接机制会引入新的竞态：例如 unlink 与 open 并发时，必须保证 open 能拿到 inode 并增加 open_count，unlink 只减少 nlinks 不立即释放；只有 nlinks==0 且 open_count==0 才释放，这对应 UNIX 的经典语义。ucore 的 inode 结构本身就维护了 open_count/ref_count (内存态)，因此链接机制的实现只需要把“链接计数”与“释放时机”纳入统一判断。

四、 实验总结

(一) 实验核心知识点与 OS 原理对应分析

通过本实验，我将操作系统原理课程中关于文件系统的抽象概念，与 ucore 中的具体实现建立了清晰对应关系。虚拟文件系统 (VFS) 通过 inode_ops 结构体，将文件操作统一抽象为一组

函数指针，使上层代码无需关心底层文件系统类型。

在 Simple FS 中，inode 负责描述文件的元数据和数据块索引，而 `sfs_io_nolock()` 则完成了从文件偏移到磁盘块访问的核心映射逻辑。通过阅读和实现该函数，我真正理解了文件系统中“逻辑上连续、物理上离散”的数据组织方式。

此外，对 `load_icode()` 的修改，使我从代码层面完整地理解了“程序即文件”这一操作系统核心思想。程序不再是特殊存在，而只是文件系统中的普通文件，通过统一的文件接口被读取并加载执行。

（二） OS 原理中重要但实验未涉及的知识点

尽管本实验实现了一个功能完整的文件系统，但仍有许多在操作系统原理中非常重要的内容未被直接涉及。例如，日志型文件系统通过写前日志（Write-Ahead Logging）来保证系统崩溃时的数据一致性，而 Simple FS 为了简化实现，并未引入相关机制。

此外，现代操作系统中普遍采用页缓存来减少磁盘访问次数，从而提升 I/O 性能，但在本实验中并未展开实现。文件系统碎片整理、延迟分配等优化策略，也未在 Simple FS 中体现。

通过 Lab8 的学习，我已经具备了理解这些高级文件系统机制所需的基础，也更加清楚地认识到课堂原理在真实系统中的实现取舍。

（三） 总结

总体来看，Lab8 是一个综合性极强的实验，它将文件系统、进程管理和虚拟内存等多个模块有机地结合在一起。通过亲自阅读和修改核心代码，我不再只是停留在概念层面，而是能够从实现角度理解操作系统各模块之间的协作关系。

本实验不仅加深了我对文件系统工作原理的理解，也让我更加体会到操作系统设计中“分层、抽象与协同”的重要性，为后续学习更复杂的操作系统机制奠定了坚实基础。